

Week 2 — Building Consensus, Bitcoin Flow, Types of Blockchains, Hash Properties, and Digital Signatures

1) Building consensus: step by step

The lecture starts with a very important idea: blockchain is not just “someone sends money and magically it appears in the chain.” There is a **process**. The slide says:

- If Alia wants to pay Babar, she launches the transaction.
- Validators put unconfirmed transactions in their **LOG** after validation.
- A miner or record producer creates the “potential” next block.
- The potential next block is verified and made part of the **journal**.
- The effort to synchronize all journals results in the **ledger**.

This is one of the most important conceptual frameworks of Week 2.

1.1 What is the big idea here?

The big idea is that a blockchain entry does not go straight from “user wants payment” to “everyone agrees.”

Instead, the data passes through multiple stages:

- first it is just a proposed transaction,
- then it sits in a temporary collection,
- then someone chooses it for a candidate block,
- then that candidate is verified,
- then nodes update their local journals,
- and only after synchronization does the shared ledger emerge.

So sir is teaching that consensus is a **pipeline**, not a single instant event.

1.2 Story version

Suppose Alia wants to pay Babar.

She uses a wallet or some blockchain application and launches the transaction.

At that moment, nothing is final yet.

The transaction is just a **proposal**. It is basically saying:
“Please update the ledger so that value moves from Alia to Babar.”

That proposed transaction is then broadcast into the network.

Different validating nodes check whether it looks valid. If it does, they place it into their local **LOG** — also called a mempool. This means:
“this transaction seems valid, but it is not yet confirmed in a block.”

Then miners or record producers look at their logs and pick some transactions from there. They package those chosen transactions into a **candidate block**.

That candidate block is then broadcast and checked by other nodes. If it satisfies protocol rules, each receiving node updates its own **journal**.

Finally, as the journals across the network synchronize, the **ledger** emerges as the agreed shared view.

So the progression is:

Transaction → Log → Record → Journal → Ledger

That exact pipeline matters a lot.

2) Data lifecycle: Stages to Consensus

The lecture formalizes the above process with these terms:

- **Transaction** — initiated and propagated by end users; any proposed change to the ledger
- **LOG** — created by auditors/validating nodes; valid but unconfirmed transactions
- **Record** — created by special nodes such as miners or record producers; selected transactions placed in a candidate record
- **Journal** — created locally by each node that receives the record, if it complies with protocol rules
- **Ledger** — emerges from the set of journals

This is extremely exam-worthy, so let's study each carefully.

2.1 Transaction

A transaction is the first step.

It is a **proposed change** to the ledger.

If Alia pays Babar, that payment request is a transaction.

Important: at the transaction stage, nothing is yet fully accepted. It is only a proposal that gets broadcast into the network.

So a transaction means:

“Here is a change I want the blockchain to record.”

2.2 LOG

After the transaction is seen and validated, it is placed in a **LOG**.

The lecture says each validating node/auditor has its own log or mempool.

This is important because beginners often imagine one giant shared pool. But that is not exactly how it works. Different nodes may have different local pools of unconfirmed transactions.

So LOG means:

- valid so far,
- not yet confirmed,
- waiting to be included in a block.

You can think of the log as a waiting room.

2.3 Record

A **record** is created by special nodes, like miners.

They select some transactions from their logs and build a **candidate record**, which basically means a candidate block.

Important detail: the slide says they may **arbitrarily select** a set of transactions.

That means miners do not have to pick transactions in one single identical order across the whole network. Different miners may choose somewhat different sets of transactions.

That is one reason temporary differences and forks can happen later.

2.4 Journal

If a node receives a candidate record and it complies with protocol rules, that node adds it to its local **journal**.

A journal is that node's own local accepted record/history.

This connects directly with Week 1, where we said every node keeps its own journal.

2.5 Ledger

The ledger is the shared agreed view that emerges from the set of journals.

This is again the same deep Week 1 idea:

the ledger is not simply one stored database file. It is the synchronized agreed state formed from many journals.

2.6 The full chain to memorize

A good one-line memory flow is:

Transaction = proposed change

Log = valid but unconfirmed holding area

Record = candidate block built by miners/record producers

Journal = local accepted history at each node

Ledger = shared converged view from all journals

That flow is very likely testable.

3) How is a block added to the blockchain?

The next slide explains the practical flow:

- a user creates a transaction,
- it goes into the pool of unconfirmed transactions,
- a miner collects new transactions,
- mining is performed,
- a block is formed if the hash condition is met,
- the block is broadcast across the network,
- other nodes validate it,
- the block gets added to the chain,
- and new blocks use the hash of the previously accepted block.

This slide also shows the rule:

$H(\text{prev hash}, \text{Tx}, \text{nonce}) < \text{Target}$

That is the Proof-of-Work condition.

3.1 What does this formula mean?

This means the miner computes a hash using:

- the previous block's hash,
- the current transactions,
- and the nonce.

The result must be smaller than some target value.

This is just a more formal way of expressing the mining difficulty condition.

Sometimes teachers simplify this as “hash must start with some zeros.”

But mathematically, the idea is that the hash output must lie under a target.

3.2 Why include previous hash?

Because the new block must be linked to the previously accepted block. This is how the chain stays connected.

3.3 Why include transactions?

Because the block is supposed to represent actual transaction data.

3.4 Why include nonce?

Because miners need something adjustable. If previous hash and transaction set are fixed, then the nonce is what miners keep changing until the hash condition is satisfied.

3.5 “Adding a subsequent block is implicit consent”

This line on the slide is subtle and important.

It means that once miners/nodes begin building the **next** block on top of a block, that behavior acts like practical acceptance of the earlier block.

In simple words:

if the network continues extending a block, it is basically saying,
“Yes, we accept this as part of the chain.”

That is why deeper blocks are more trusted: later blocks strengthen earlier ones.

4) Recap pyramid: from unconfirmed transactions to ledger

The recap slide visually shows a pyramid:

- Unconfirmed Transactions
- Log
- Records
- Journal
- Ledger

The arrow says **processing** as we move upward.

What this pyramid means

This is not just decoration. It shows that consensus is a progression from raw proposals to fully accepted shared state.

At the bottom are many candidate changes — unconfirmed transactions.

Then they move upward through filtering, validation, packaging, local acceptance, and finally shared convergence.

So the system gradually refines uncertainty into agreement.

That is the whole spirit of consensus.

5) Characteristics of an ideal blockchain / DLT

The lecture lists the following desirable properties:

- Shared recordkeeping
- Multi-party consensus
- Independent validation
- Tamper evidence
- Tamper resistance

These are very important because sir may ask: “What makes a good blockchain system?”

Let’s study each.

5.1 Shared recordkeeping

Multiple entities can write data.

This means the system is not dependent on just one writer or one central organization. Many participants contribute entries.

This matters because blockchain is supposed to support a shared system across multiple parties.

5.2 Multi-party consensus

All parties must come to agreement.

This means not just one node deciding truth for everyone else. The accepted state must reflect agreement across the distributed participants.

5.3 Independent validation

Every fully validating node should be able to recompute and verify the entire system state.

This is a very powerful property.

It means trust is reduced because any honest full validator can check things itself instead of blindly trusting another party's statement.

This is one of the deepest strengths of blockchain.

5.4 Tamper evidence

Participants should be able to easily detect unauthorized changes.

For example, if someone changes old transaction data, the hashes will no longer match correctly, and the chain structure exposes the alteration.

So tamper evidence means:
if someone cheats, it is visible.

5.5 Tamper resistance

This goes one step beyond evidence.

It means it should not just be detectable — it should also be **hard** to successfully change past history.

In other words:
tamper evidence = we can notice cheating
tamper resistance = cheating is costly/difficult to carry out

That difference is important.

6) Byzantine Generals Problem

Now the lecture gives a famous historical computing analogy: the **Byzantine Generals Problem**.

The slide says:

- several divisions of the Byzantine army are camped outside an enemy city,
- each division has its own general,
- they must agree on a common battle plan,
- but some may be traitors and may prevent loyal generals from reaching agreement.

This is one of the most famous concepts in distributed systems.

6.1 What is the real point of this story?

The story is not actually about armies.

It is a metaphor for distributed systems.

The real question is:

How can multiple independent participants reach agreement when communication may be unreliable and some participants may be faulty or malicious?

That is exactly the blockchain problem too.

6.2 Why collective action matters

The lecture says if generals attack collectively, they will win; otherwise they may lose.

This means partial agreement is not enough.

If some attack and others withdraw, the result is disaster.

So the challenge is not just sending messages.

The challenge is reaching one reliable common decision.

6.3 What can go wrong?

The slide lists messenger problems:

- the message may be intercepted,
- it may reach only some generals,
- it may be relayed incorrectly,
- it may be forged,
- a malicious general may deliberately send wrong information.

This is teaching that distributed communication has many failure modes: loss, inconsistency, deception, and bad actors.

That is exactly why consensus is hard.

7) Byzantine faults in computing systems

The lecture then connects the generals story to real computing. It says decentralized systems may encounter Byzantine faults such as:

- broken hardware
- congested network
- disconnected nodes
- incorrect processing of requests
- corrupted local states
- nodes producing inconsistent output
- malicious coordinated attacks

7.1 What is a Byzantine fault?

A Byzantine fault is not just a simple crash.

It is a situation where a component behaves in an inconsistent, unpredictable, or malicious way.

A node may:

- send wrong messages,
- behave differently toward different peers,
- hold corrupted state,
- or intentionally deceive.

So Byzantine faults are the hardest kind of failures because they are not neat and obvious.

7.2 Byzantine fault tolerance

The lecture defines Byzantine fault tolerance as the ability to prevent or tolerate Byzantine failures.

So if a system is Byzantine-fault tolerant, it can still work correctly even if some participants are unreliable or malicious.

7.3 Why Bitcoin is important here

The slide says Bitcoin was the first blockchain to solve consensus for an **unknown number of generals**.

That means Bitcoin showed how open participants in a permissionless environment could still converge on one ledger without needing a fixed pre-approved membership list.

That is historically huge.

8) Techniques used in Bitcoin Blockchain

The lecture lists four important techniques:

- Public-key cryptography
- Atomic broadcast of all transactions
- Proof of Work
- Chaining of blocks

Let's understand what each contributes to the generals problem.

8.1 Public-key cryptography

Used for unforgeable digital signatures from authenticated sources.

The slide says this authenticates generals and prevents messengers from forging.

In blockchain language, that means:

- the sender can prove the transaction really came from them,
- others cannot easily fake that authorization.

So this solves the authenticity problem.

8.2 Atomic broadcast of all transactions

The slide says no general should miss receiving the transaction.

This means the system tries to propagate transactions widely so the network can reason over the same candidate information.

8.3 Proof of Work

PoW makes deception expensive.

If malicious actors want to rewrite history or dominate consensus, they must pay a very high computational and energy cost.

So PoW does not magically stop dishonesty, but it makes dishonesty costly.

8.4 Chaining

Every new block further cements the past.

This means once transactions are buried under more and more blocks, altering them becomes harder and harder because the attacker must redo all later work too.

So chaining helps make history increasingly stable.

9) How does the blockchain get built up? Longest chain, forks, and finality

The lecture says:

- the longest chain is picked,
- forks can occur,
- a transaction becomes final only after multiple confirmations,
- because forks can temporarily reorder or exclude it.

This is a very important cluster of concepts.

9.1 Why might there be a choice between chains?

Sometimes two miners produce valid blocks at nearly the same time.

Then different parts of the network may hear about different blocks first.

As a result, for a short time, the network may not have one perfectly unified chain tip. Instead, there may be two competing versions.

That situation is called a **fork**.

9.2 What is a fork?

A fork is when there are temporarily different views of the journal/history.

In simple words:

for a short time, some nodes think one block is the latest valid block, while others think another block is.

9.3 Why pick the longest chain?

In Bitcoin-style consensus, the chain with the greatest accumulated work — usually simplified in class as the “longest chain” — is treated as the winning chain.

Why?

Because it represents the branch the network has continued building on most strongly.

So the system resolves temporary disagreement by following the chain that gets extended further.

9.4 What is finality?

Finality means:

when can we treat a transaction as practically settled and unlikely to be reversed?

The lecture says a transaction becomes final only after **multiple confirmations**.

A confirmation means additional blocks have been built after the block containing that transaction.

9.5 Why are multiple confirmations needed?

Because forks can temporarily reorder or exclude a transaction.

If your transaction is in a block that later loses the fork competition, that transaction may disappear from the main chain and need re-inclusion.

So one confirmation is not always enough for strong confidence.

The more blocks are added on top, the more stable the transaction becomes.

This is a really important exam concept:

confirmation depth increases confidence.

10) Types of blockchain

The lecture now shifts to classify blockchains.

It says **permissionless blockchains**:

- must be public,
- examples: Bitcoin and Ethereum,
- anyone can join or leave at any time,
- no central entity controls membership,
- and anyone can read.

Then it discusses **permissionless vs permissioned**, and public vs private combinations.

Let's study this carefully.

10.1 Permissionless blockchain

Permissionless means you do not need approval from a central authority to join as a participant.

Anyone can enter or leave the system.

Examples:

- Bitcoin
- Ethereum

This fits the original blockchain dream of open decentralized participation.

10.2 Why must permissionless be public?

Because if anyone can join and there is no central membership manager, then the system must be open for reading and participation in a public way.

10.3 Privacy and anonymity in permissionless chains

The lecture says:

- privacy is compromised by design because auditability is needed,
- anonymity is feasible with cryptographic procedures.

This is a subtle but important distinction.

Privacy means hidden data.

Anonymity means identities may be obscured.

So a blockchain may be publicly readable but still use cryptographic addresses rather than real names. That gives some degree of anonymity, but not full privacy of transaction flow.

10.4 Permissioned blockchain

In permissioned systems, some or all writing rights are restricted to authorized nodes.

The slide's table says:

- **Public permissionless:** anyone can read or write
- **Public permissioned:** anyone can read, but only authorized nodes can write
- **Private permissioned:** read/write depend on authorization

So permissioned means membership and/or writing rights are controlled.

These systems are often used where organizations want some blockchain benefits but not full public openness.

11) When not to use a blockchain

This is a super practical slide. It asks a sequence of questions:

- Need to store state?
- Multiple writers?
- Online trusted third party always available?
- All writers known?
- All writers trusted?
- Public verifiability needed?

Then it routes to:

- permissionless blockchain,
- public permissioned,
- private permissioned,
- or **do not use blockchain.**

11.1 What is the main lesson?

The main lesson is:

blockchain is not the answer to every data problem.

Blockchain is only worth the complexity if:

- there are multiple writers,
- trust is limited,
- public/shared verifiability matters,
- and a central always-trusted authority is not sufficient.

11.2 When a normal database is enough

If one trusted online party is always available and can manage the data fairly, then often a normal database is better.

This is very important because it shows intellectual maturity: blockchain is a specialized tool, not a universal magic solution.

12) Cryptocurrencies as a blockchain use case

The lecture then clarifies the Bitcoin-blockchain relationship.

It says:

- miners “mine” by creating and adding blocks, not by digging like gold,
- bitcoin (BTC) is a digital cryptocurrency tradeable on its own blockchain,
- miners are rewarded with bitcoins when they create an accepted block,
- and new bitcoins come from the protocol, which simply writes a transaction on the blockchain.

12.1 What do miners really mine?

Miners are not mining coins out of the ground.

They are mining **blocks**.

That means they perform the work required to create a valid block and append it to the chain.

12.2 Then what is BTC?

BTC is the digital asset that lives on the Bitcoin blockchain.

Ownership and transfer of BTC are recorded by transactions on that blockchain.

12.3 Where do new bitcoins come from?

This is very important.

The system itself creates them through the protocol as part of block creation.

So the protocol can include a special transaction rewarding the miner of a newly accepted block.

This is called minting via the protocol, not physical extraction.

13) Bitcoin transaction: step by step

Lecture 04 begins the practical Bitcoin flow.

It says:

- user initiates transaction from wallet or exchange app,
- transaction is broadcast,
- it is placed in a pool of unconfirmed transactions,
- there are multiple local pools rather than one giant pool,
- miners wait to pick them up,
- miners are incentivized by subsidy rewards plus transaction fees.

Let's go through the full lifecycle carefully.

13.1 User initiates transaction

The sender creates a transaction through software such as a wallet.

This is the user saying:

"I want to transfer BTC."

13.2 Broadcast

The transaction is sent into the network for execution/consideration.

At this moment, it is still not confirmed.

13.3 Unconfirmed transaction pools

The transaction enters pools of unconfirmed transactions.

Important detail:

there is not one single giant universal pool. Different nodes maintain local mempools.

13.4 Why would miners care?

Because miners earn rewards:

- block subsidy
- transaction fees

So mining has an economic incentive.

14) Bitcoin transaction: miners pick and solve

The next slide says:

- one transaction may be picked by multiple miners,
- one miner may pick multiple transactions into one block,
- Bitcoin block size is about ~1MB,
- miners search over the nonce space,
- an eligible hash is hard to find but easy to verify,
- and a correct hash is the Proof of Work.

14.1 Many-to-many paradigm

This is important.

A transaction is not assigned to exactly one miner.

Multiple miners may independently choose the same unconfirmed transaction.

At the same time, each miner usually includes many transactions in the block they are trying to mine.

14.2 Candidate block

The miner forms a block candidate containing:

- chosen transactions,
- previous hash,
- nonce,
- and other required metadata.

14.3 Search over nonce space

The miner tries different nonce values until the block hash satisfies the target condition.

This is the computational search process of mining.

14.4 Hard to find, easy to verify

This is the essence of PoW.

Finding the valid hash takes many trials.
But once someone finds one, other nodes can verify it quickly.

That asymmetry is what makes PoW useful.

15) Bitcoin transaction: broadcasting solution and confirmation

The next slide says:

- successful miner broadcasts block + hash,
- other miners verify it,
- consensus agrees,
- block is added to their own journals,
- additional blocks after it give confirmations,
- block subsidy becomes spendable after 100 confirmations,
- and on receiving a new accepted block, miners must start afresh using the new latest hash.

15.1 Broadcasting the solution

The winning miner shares:

- the proposed block,
- the proof that its hash satisfies the rule.

15.2 Easy verification

Other nodes quickly verify the hash and the block contents.

This is why PoW works well operationally:
expensive to produce, cheap to check.

15.3 Added to journals

Notice the lecture carefully says block is added to the blockchain — really, to each node's own journal first.

That matches the DLT idea that the ledger is the synchronized outcome of many journals.

15.4 Confirmations

If more blocks are added after this block, confidence increases that it will remain in the main chain.

15.5 Why miners must start afresh

Because the next block must include the hash of the latest accepted block.

So when a new block wins, the entire mining target context changes. Continuing to mine on an outdated parent would waste effort on a branch less likely to survive.

16) Bitcoin ownership and transactions

The lecture says:

- BTC is always **locked** to the owner,
- in a transaction the payer unlocks it and then locks it again for the payee,
- new BTC is minted only at block creation,
- and gets locked to the creator of the block.

This is a very important way of thinking.

16.1 BTC is not just “inside an account” in a casual sense

The lecture frames BTC as being cryptographically locked to its owner.

That means ownership is controlled through cryptographic rights, not just through a username/password model.

16.2 Transaction = unlock then relock

The payer must prove the right to spend.

So they unlock value that was previously locked to them.

Then the transaction creates new locking conditions so that only the payee can later unlock/spend it.

This is a really nice mental model for blockchain transfers.

16.3 Minting happens only at block creation

New BTC is not randomly created any time by any user.

The protocol mints new BTC only through the special reward transaction associated with accepted block creation.

17) BTC and fiat

The lecture gives a historical fact:

The New Liberty Standard Exchange recorded an early BTC-to-fiat exchange in late 2009 where 5,050 bitcoins were traded for \$5.02, about \$0.00099 per BTC. It contrasts that with an average BTC price around \$89,000 in Jan 2026.

Why is this included?

This is probably here to show:

- BTC began as something with tiny fiat value,
- over time markets assigned exchange value to it,
- and the relation between crypto and fiat emerged historically through real trades.

This is less concept-heavy than other parts, but still important context.

18) Hash function properties

The lecture now deepens hashing beyond Week 1. It says a cryptographic hash function has:

- arbitrary size input
- fixed size output
- random-looking but deterministic output
- roughly half ones and half zeros
- avalanche effect
- preimage resistance
- collision resistance

This is very important and highly testable.

18.1 Deterministic but random-looking

This is a great exam distinction.

A hash output **looks random**, but it is actually deterministic.

That means:
same input → always same output

So it is not true randomness. It only appears random-looking.

18.2 Balanced bit pattern

The slide says about half the bits are ones and half are zeros.

This means good hash outputs should look statistically well-distributed, not biased or patterned.

18.3 Avalanche effect

If one bit in the input changes, about half the output bits change.

This is an excellent property because it makes tiny input changes produce dramatically different outputs, which helps tamper detection.

18.4 Preimage resistance

Given a hash value Y , it should be computationally infeasible to find some X such that $\text{Hash}(X) = Y$.

In simple words:

if I show you the output hash, you should not be able to reverse-engineer an input that produced it.

18.5 Collision resistance

It should be infeasible to find two different inputs X and Z where $X \neq Z$ but $\text{Hash}(X) = \text{Hash}(Z)$.

In simple words:

two different messages should not easily land on the same hash.

This is extremely important for security.

19) What is needed for a transaction? Compare with a check

The lecture compares blockchain transactions with a check. It lists:

- payer
- payee

- amount
- authorization
- can the check be honored?
- does the account have enough?
- account balance checked offline

19.1 Why this analogy is used

A check is familiar. It helps explain what a transaction needs in general.

Any valid value transfer needs:

- sender,
- receiver,
- amount,
- proof that the sender approved it.

19.2 “Can this check be honored?”

This means:

does the payer really have enough money?

So validity is not just about signatures; it is also about funds availability.

19.3 Blockchain version

In blockchain too, a valid transaction needs:

- payer identity/address,
- payee identity/address,
- amount/value,
- authorization via digital signature,
- and some way to ensure the payer has spendable funds.

This is setting up the need for digital signatures.

20) Digital signatures

The lecture says digital signatures are:

- like manual signatures, but different for each occurrence,
- needed to authenticate the sender,
- needed to avoid forgery,

- and the receiver/address must also be identified.

This is one of the most important foundations of blockchain transactions.

20.1 What is a digital signature?

A digital signature is a cryptographic proof attached to a message/transaction that shows it was authorized by the holder of the correct private key.

It is like signing, but not with ink.

20.2 Why “different for each occurrence”?

Because digital signatures are not meant to be one fixed picture copied again and again like a handwritten signature on paper might be.

They are generated cryptographically for a specific message using the sender’s private key.

So they are tied to that particular message occurrence.

20.3 Why are digital signatures needed?

Two main reasons:

- they authenticate the sender,
- they prevent message forgery.

If someone claims a payment came from Alice, the network needs cryptographic proof that Alice really authorized it.

Without digital signatures, anyone could falsely claim to send money from anyone else.

21) What is cryptography? Symmetric cryptography first

The lecture then explains cryptography through the goal:
send a message that a spy cannot understand.

It introduces **symmetric key cryptography**, where the same shared key is used for encryption and decryption, and says the big challenge is **key sharing**. It also asks: if Bob receives some data, can he be sure it came from Alice?

21.1 Basic goal of cryptography

Cryptography tries to protect communication and data using mathematical techniques.

In this slide, the initial focus is confidentiality:
a spy should not be able to make sense of the message.

21.2 Symmetric key cryptography

In symmetric cryptography:

- Alice and Bob share the same secret key.
- Alice encrypts with that key.
- Bob decrypts with that same key.

This works for secrecy if the shared key remains secret.

21.3 The big problem: key sharing

How do Alice and Bob safely share that secret key in the first place?

If the key is intercepted during sharing, the whole system breaks.

That is why symmetric systems face a major key distribution challenge.

21.4 Another limitation: source authentication

If Bob receives encrypted data and successfully decrypts it, can he be completely sure it came from Alice?

Not necessarily — if others know the shared key, attribution is unclear.

That is why blockchain needs stronger sender-specific authorization methods, which leads to public key cryptography and digital signatures.

22) Public Key Infrastructure and Digital Signatures

The lecture gives three functions:

- **GenerateKeys()** → produces publicKey and privateKey
- **Sign(message, privateKey)** → sender creates signature
- **Verify(message, signature, publicKey)** → receiver checks validity; output is Boolean

This is absolutely core.

22.1 GenerateKeys()

A user generates a key pair:

- one public key
- one private key

The private key must be kept secret.

The public key can be shared.

22.2 Sign(message, privateKey)

The sender uses the private key to create a signature on the message.

That signature proves authorization by the holder of the private key.

The signature is sent along with the message.

22.3 Verify(message, signature, publicKey)

Anyone with the public key can verify whether:

- this signature matches the message,
- and whether it was produced by the corresponding private key.

The result is yes/no — verified or not.

22.4 Why this is perfect for blockchain

Because blockchain needs public verifiability and secure authorization.

A transaction can be broadcast publicly, and anyone can verify its signature using the sender's public key — without learning the private key.

That is brilliant, and that is why digital signatures are foundational.

Week 2 — Full connected story

Now let me connect the whole week as one story.

Consensus in blockchain is not one single instant. A user first creates a transaction, which is just a proposed change to the ledger. Validating nodes check such transactions and place valid

but still unconfirmed ones into their local logs or mempools. Special nodes such as miners then select some transactions from their logs and package them into a candidate record/block. If that block satisfies the protocol rules, nodes add it to their local journals. As journals across the network synchronize, the shared ledger emerges. That is how consensus is built step by step.

A valid block in Proof of Work must satisfy a hash condition of the form $H(\text{previous hash, transactions, nonce}) < \text{target}$. Miners therefore search over nonce values until a valid hash is found. When a block is extended by later blocks, that acts as implicit consent and strengthens confidence in it. This is why confirmations matter.

An ideal blockchain supports shared recordkeeping, multi-party consensus, independent validation, tamper evidence, and tamper resistance. But achieving consensus in a distributed system is hard, as illustrated by the Byzantine Generals Problem: multiple independent actors must agree, even though communication may be unreliable and some actors may be malicious. In computing, such problems appear as Byzantine faults — inconsistent, faulty, or malicious behavior by nodes. Bitcoin addresses this using public key cryptography, widespread broadcast, Proof of Work, and block chaining.

Because different parts of the network may temporarily see different valid chain tips, forks can occur. Bitcoin resolves this by following the longest chain, and transactions are treated as final only after multiple confirmations, since temporary forks may reorder or exclude them.

The lecture then classifies blockchains into permissionless and permissioned forms. Permissionless chains are public and open to anyone, while permissioned chains restrict writing and sometimes reading to authorized participants. This leads to the practical lesson that blockchain should only be used when there are multiple writers, limited trust, and a real need for shared verifiability without relying on one always-available trusted party.

Bitcoin is then used as the example blockchain application. Users create transactions, miners collect them, solve a Proof-of-Work puzzle, broadcast the block, and other nodes verify it and update their journals. BTC itself is understood as value locked to an owner; transactions unlock it from the payer and lock it again to the payee. New BTC is minted only through block creation rewards defined by the protocol.

Finally, the week deepens the security foundations. Hash functions take arbitrary-length input and produce fixed-size deterministic but random-looking output with avalanche effect, preimage resistance, and collision resistance. Transactions, like checks, require a payer, payee, amount, authorization, and funds validity. Digital signatures provide sender authentication and anti-forgery protection using public key cryptography: keys are generated, messages are signed with the private key, and signatures are verified with the public key. This is how blockchain transactions prove authorization in a public network.

Most important Week 2 topics to memorize

These are the biggest ones:

- Transaction → Log → Record → Journal → Ledger
 - How a block is added to blockchain
 - Proof-of-Work target condition
 - Byzantine Generals Problem
 - Byzantine faults
 - Bitcoin techniques: signatures, broadcast, PoW, chaining
 - Forks, longest chain, finality, confirmations
 - Permissionless vs permissioned
 - When not to use blockchain
 - Bitcoin transaction flow
 - BTC ownership as locking/unlocking
 - Hash properties: avalanche, preimage resistance, collision resistance
 - Digital signatures and public key infrastructure
-

Ultra-short ratta revision for Week 2

Consensus is built in stages: a user creates a transaction, validating nodes keep valid unconfirmed transactions in logs, miners create candidate records/blocks, accepted records enter node journals, and the shared ledger emerges from synchronized journals. In PoW, miners solve for a hash under a target by changing the nonce. An ideal blockchain has shared recordkeeping, multi-party consensus, independent validation, tamper evidence, and tamper resistance. The Byzantine Generals Problem explains why distributed consensus is hard when some participants or messages may be faulty or malicious. Bitcoin addresses this using public key cryptography, wide transaction broadcast, Proof of Work, and chaining. Forks create temporary competing histories, so the longest chain is followed and finality needs multiple confirmations. Blockchains may be permissionless or permissioned, public or private. Bitcoin transactions move from wallet → broadcast → mempool → miner selection → PoW → broadcast of solution → verification → confirmations. BTC is locked to owners and transferred by unlocking then relocking. Hash functions are fixed-size, deterministic, avalanche-producing, preimage-resistant, and collision-resistant. Digital signatures authenticate senders using public/private keys.